МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №3

"Створення та моделювання баз даних в MongoDB"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

ас. Жишкович А. В.

Львів-2026

Мета роботи: Ознайомлення з роботою СКБД MongoDB у консолі mongosh. Отримання практичних навичок створення бази даних, звичайних та обмежених (capped) колекцій і документів, їх вилучення та перейменування. Відпрацювання базових CRUD-операцій і практичне моделювання зв'язків між документами за трьома типами: вкладені документи (Embedded Documents), посилання (References через ObjectId) та серверне об'єднання $lookup.

Теоретичні відомості:

MongoDB є документо-орієнтованою нереляційною системою керування базами даних (NoSQL). Замість таблиць реляційних СКБД у MongoDB використовуються колекції, а замість рядків - документи у двійковому форматі BSON (Binary JSON). Це забезпечує гнучку динамічну схему даних (schemaless), коли документи в одній колекції можуть мати різний набір полів і типів.

Архітектура роботи побудована за моделлю «клієнт-сервер»:
1. Сервер mongod - основний фоновий демон СУБД, що управляє пам'яттю, файлами сховища WiredTiger та сокетом на порті 27017.
2. Клієнт mongosh - інтерактивна оболонка на рушії Node.js, яка підтримує повноцінний синтаксис JavaScript, автодоповнення та автоматичне форматування виводу.

Окремим типом колекцій є обмежені колекції (capped collections). Це кільцеві буфери суворо фіксованого розміру (size у байтах), де при переповненні найстаріші документи автоматично вилучаються за принципом FIFO (First-In, First-Out). Точкове видалення окремих документів з capped collections заборонене.

Для організації зв'язків між сутностями в MongoDB застосовують три основні підходи:
• Вкладені документи (Embedded Documents) - денормалізація даних, при якій залежні об'єкти чи масиви зберігаються всередині одного документа. Забезпечує максимальну швидкість читання та атомарність без використання операцій JOIN.
• Посилання (References) - нормалізоване збереження у полі документа 12-байтного унікального ідентифікатора _id (ObjectId) іншого документа. Запобігає дублюванню інформації та переповненню максимального розміру документа (16 МБ).
• $lookup (конвеєр агрегації) - етап агрегації, що реалізує аналог лівого зовнішнього з'єднання (LEFT OUTER JOIN) на стороні сервера.

Виконання роботи

Таблиця 1. Порівняння основних концепцій реляційних СУБД та MongoDB

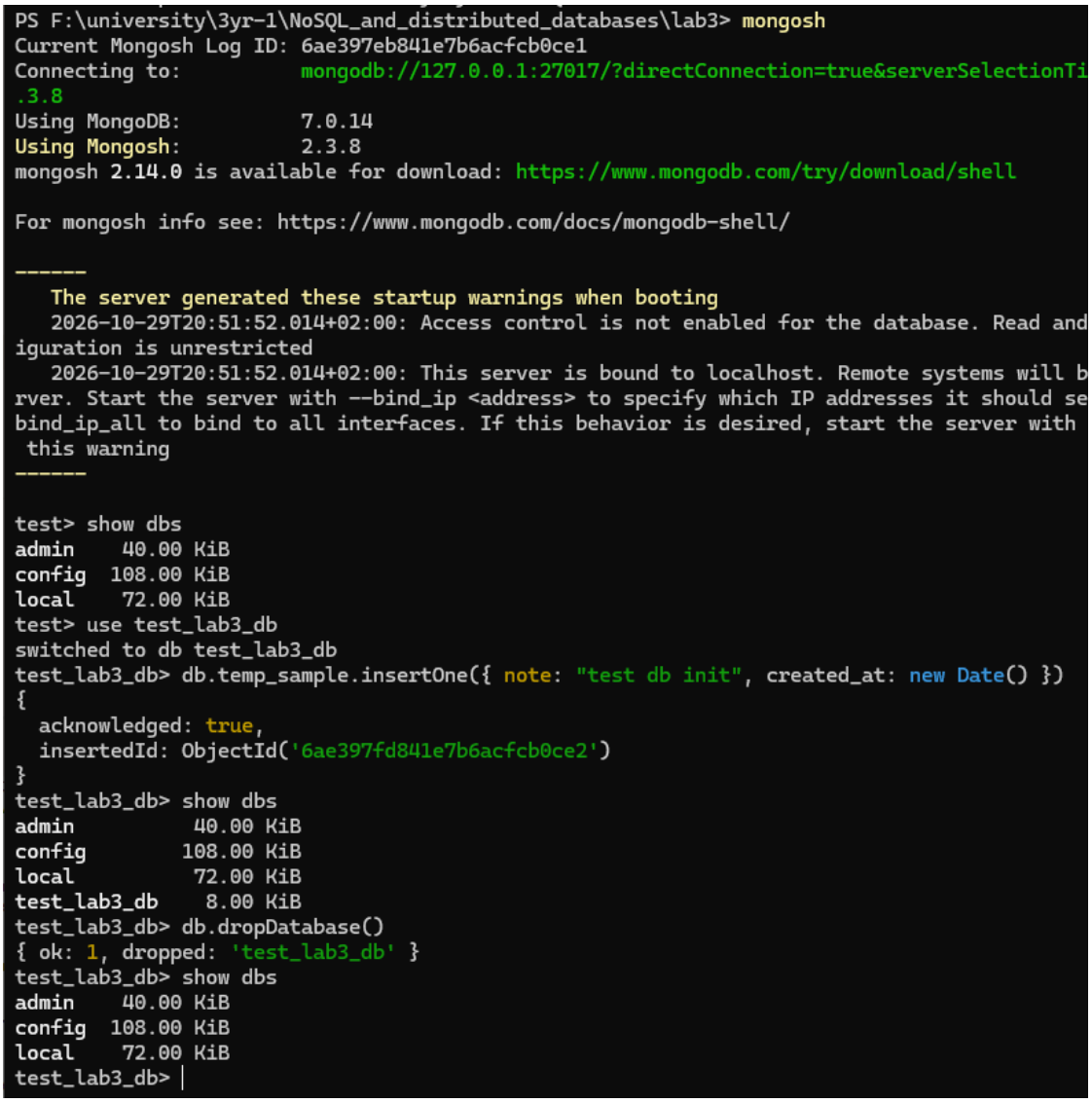
Реляційні СКБД (SQL)	MongoDB (NoSQL Document)	Порівняльна характеристика
База даних (Database)	База даних (Database)	Фізичний контейнер для логічного набору просторів даних
Таблиця (Table)	Колекція (Collection)	Набір документів; відсутня вимога жорсткої фіксованої схеми
Рядок (Row / Tuple)	Документ (BSON Document)	Одиниця збереження даних у форматі пар ключ-значення
Стовпець (Column)	Поле (Field)	Атрибут документа з динамічною типізацією значень
Первинний ключ (Primary Key)	Поле _id (ObjectId)	12-байтний глобально унікальний системний ідентифікатор
З'єднання (JOIN)	Embedded або $lookup	Замінюється вкладенням (денормалізація) або агрегацією $lookup

Таблиця 2. Порівняльний аналіз підходів до моделювання зв'язків у MongoDB

Тип зв'язку	Механізм реалізації	Головні переваги	Недоліки та обмеження
Embedded Documents	Вкладений піддокумент або масив об'єктів	Максимальна швидкість читання, атомарність оновлень, відсутність JOIN	Ліміт 16 МБ на документ, ризик дублювання даних
References (ObjectId)	Збереження поля з _id документа іншої колекції	Нормалізованість даних, відсутність дублювання, необмежений обсяг	Потребує кількох запитів до бази або виконання конвеєра $lookup
$lookup Aggregation	Етап агрегації з полями from, localField, foreignField	Об'єднання колекцій на стороні сервера, гнучка аналітична фільтрація	Вищі витрати ресурсів CPU та оперативної пам'яті сервера

1. Запустив локальний сервер mongod та підключився до нього через клієнтський інтерфейс mongosh у терміналі. Переглянув початковий перелік системних баз даних командою 'show dbs' (відображено лише системні admin, config, local). Для перевірки життєвого циклу бази даних створив тестову БД 'test_lab3_db' викликом 'use test_lab3_db', ініціалізував її дисковий простір додаванням документа методом 'insertOne()', переконався у появі бази в 'show

dbs' (розмір 8.00 KiB), після чого повністю вилучив її викликом 'db.dropDatabase()' і підтвердив видалення повторним викликом 'show dbs':



2. Перемкнувся на базу даних предметної області 'use techstore'. Створив стандартну колекцію 'audit_logs' командою 'db.createCollection("audit_logs")'. Створив обмежену кільцеву колекцію 'cartoons' із фіксованим лімітом розміру 5 МБ (size: 5242880) та максимум 5000 документів (max: 5000). Методом 'db.cartoons.isCapped()' перевірив статус обмеження, отримавши підтвердження 'true'. Перейменував колекцію на 'animated_films' за допомогою 'renameCollection("animated_films")' та перевірив перелік колекцій 'show collections'. Після завершення перевірки вилучив створені тестові колекції

викликами 'db.animated_films.drop()' та 'db.audit_logs.drop()', підтвердивши порожній стан викликом 'show collections':

```
test_lab3_db> use techstore
switched to db techstore
techstore> db.createCollection("audit_logs")
{ ok: 1 }
techstore> db.createCollection("cartoons", { capped: true, size: 5242880, max: 5000 })
{ ok: 1 }
techstore> db.cartoons.isCapped()
true
techstore> db.cartoons.renameCollection("animated_films")
{ ok: 1 }
techstore> show collections
animated_films
audit_logs
techstore> db.animated_films.drop()
true
techstore> db.audit_logs.drop()
true
techstore> show collections

techstore>
```

3. Для виконання практичного циклу робіт обрав предметну область 'Магазин комп'ютерної техніки' (база даних 'techstore'). Спроєктував логічну схему даних із трьома взаємопов'язаними колекціями: 'products' (каталог комплектуючих), 'customers' (покупці) та 'orders' (замовлення). Кожен документ містить від 6 до 8 полів різноманітних BSON-типів. Структуру полів зведено у таблицю 3:

Таблиця 3. Схема даних колекцій предметної області TechStore

Колекція	Поле	Тип BSON	Призначення атрибута
products	_id	ObjectId	Унікальний первинний ідентифікатор товару
products	name	String	Повна назва комп'ютерного комплектуючого
products	category	String	Категорія товару (Процесори, Відеокарти тощо)
products	price	Double / Int	Роздрібна ціна товару в гривнях
products	in_stock	Boolean	Прапорець фактичної наявності на складі
products	tags	Array[String]	Масив пошукових тегів комплектуючого
products	specifications	Embedded Doc	Вкладені характеристики (socket, vram, ram_type)
customers	_id	ObjectId	Унікальний первинний ключ клієнта
customers	customer_code	String	Унікальний бізнес-код покупця (CUST-001)
customers	name	String	Прізвище та ім'я покупця
customers	phone	String	Контактний номер мобільного телефону
customers	address	Embedded Doc	Вкладений піддокумент адреси (city, street, zip)
orders	_id	ObjectId	Унікальний ідентифікатор товарного чека
orders	order_number	String	Номер замовлення (ORD-2026-101)

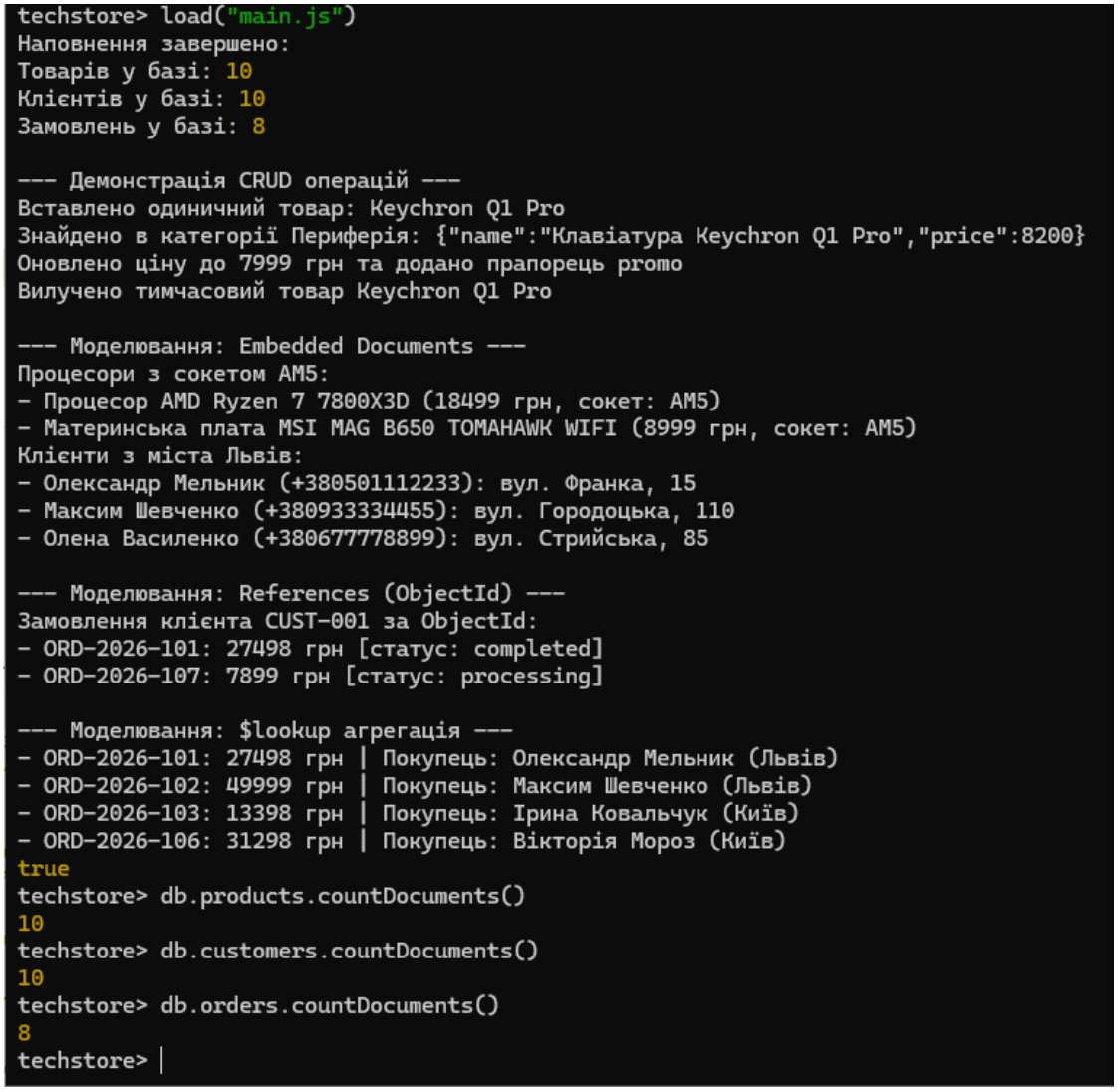
orders	customer_id	ObjectId (Ref)	Зовнішнє посилання на _id клієнта в customers
orders	order_date	Date	Точна дата та час оформлення замовлення
orders	status	String	Статус виконання (completed, processing, shipped)
orders	total_amount	Double / Int	Загальна розрахункова вартість чека в грн
orders	items	Array[Doc]	Вкладений масив товарних позицій (product_id, назва, ціна)

Підготував скрипт первинного наповнення 'main.js' і виконав його безпосередньо в інтерактивній сесії mongosh командою 'load("main.js")'. Скрипт успішно завантажив 10 товарів, 10 покупців та 8 замовлень. Перевірив кількість збережених документів у кожній колекції за допомогою методу 'countDocuments()':

```
// NoSQL_and_distributed_databases/lab3/main.js
db = db.getSiblingDB("techstore");

// наповнення колекції товарів (10 документів, різні типи BSON)
db.products.insertMany([
  {
    name: "Процесор AMD Ryzen 7 7800X3D",
    category: "Процесори",
    price: 18499.0,
    in_stock: true,
    stock_quantity: 25,
    tags: ["cpu", "am5", "gaming", "v-cache"],
    specifications: {
      socket: "AM5",
      cores: 8,
      threads: 16,
      base_clock_ghz: 4.2,
      tdp_watts: 120,
      ram_type: "DDR5"
    }
  },
  {
    name: "Відеокарта ASUS ROG Strix GeForce RTX 4080 Super",
    category: "Відеокарти",
    price: 52999.0,
    in_stock: true,
    stock_quantity: 7,
    tags: ["gpu", "nvidia", "rtx", "4k-gaming"],
    specifications: {
      vram_gb: 16,
      bus_width_bit: 256,
      interface: "PCI-e 4.0",
      recommended_psu_watts: 750
    }
  }
  // ... ще 8 документів (материнські плати, пам'ять, SSD, БЖ, монітори)
]);

// ... наповнення db.customers (10 документів з вкладеними адресами)
// ... наповнення db.orders (8 документів з посиланнями на ObjectId)
```



4. Відпрацював базові операції маніпулювання даними (CRUD) безпосередньо у середовищі mongosh:

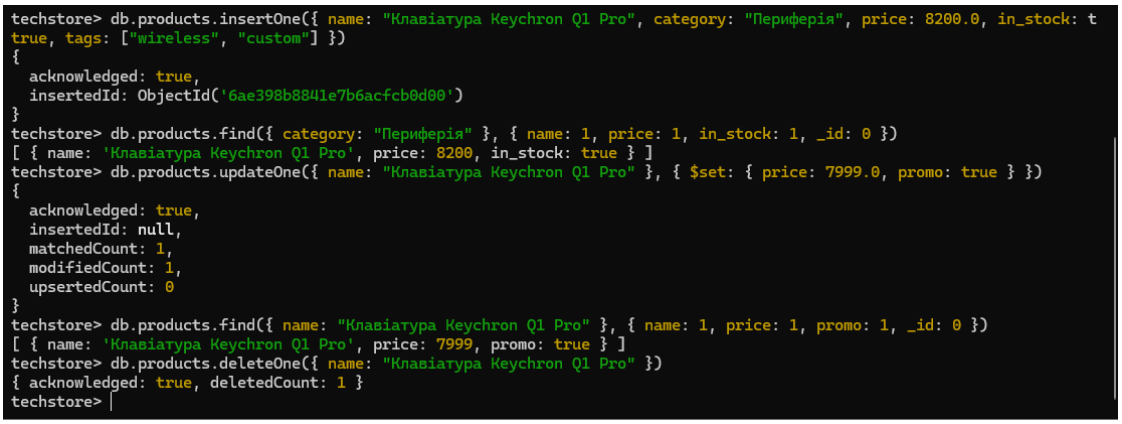
• Create: додав окремий новий товар 'Клавіатура Keychron Q1 Pro' у категорію 'Периферія' викликом 'insertOne()';

• Read: виконав пошук доданого документа через 'find()' із проєкцією полів name, price, in_stock;

• Update: оновив ціну товару з 8200 грн до 7999 грн та встановив прапорець 'promo: true' атомарним оператором '$set' у 'updateOne()';

• Read (перевірка): переконався у внесенні змін повторним запитом 'find()';

• Delete: вилучив тестовий документ за допомогою 'deleteOne({ name: "..." })' (отримано deletedCount: 1):



5. Реалізував першу обов'язкову стратегію моделювання зв'язків: Вкладені документи (Embedded Documents). У колекції 'products' технічні параметри виділено у піддокумент 'specifications', а в 'customers' адресу оформлено у піддокумент'address'. Виконав три пошукові запити до вкладених структур із застосуванням крапкової нотації (dot-notation):

1) Пошук комплектуючих на платформі сокета AM5 ('specifications.socket': 'AM5');

2) Пошук покупців із міста Львів за атрибутом 'address.city': 'Львів';

3) Комплексний пошук комплектуючих стандарту DDR5 за ціною менше 10 000 грн (запит коректно повернув материнську плату MSI та пам'ять Kingston Fury):

```
techstore> db.products.find({ "specifications.socket": "AM5" }, { name: 1, price: 1, "specifications.s
ocket": 1, _id: 0 })
[
  {
    name: 'Процесор AMD Ryzen 7 7800X3D',
    price: 18499,
    specifications: { socket: 'AM5' }
  },
  {
    name: 'Материнська плата MSI MAG B650 TOMAHAWK WIFI',
    price: 8999,
    specifications: { socket: 'AM5' }
  }
]
techstore> db.customers.find({ "address.city": "Львів" }, { name: 1, phone: 1, "address.street": 1, _i
d: 0 })
[
  {
    name: 'Олександр Мельник',
    phone: '+380501112233',
    address: { street: 'вул. Франка, 15' }
  },
  {
    name: 'Максим Шевченко',
    phone: '+380933334455',
    address: { street: 'вул. Городоцька, 110' }
  },
  {
    name: 'Олена Василенко',
    phone: '+380677778899',
    address: { street: 'вул. Стрийська, 85' }
  }
]
techstore> db.products.find({ "specifications.ram_type": "DDR5", price: { $lt: 10000 } }, { name: 1, p
rice: 1, "specifications.ram_type": 1, _id: 0 })
[
  {
    name: 'Материнська плата MSI MAG B650 TOMAHAWK WIFI',
    price: 8999,
    specifications: { ram_type: 'DDR5' }
  },
  {
    name: "Оперативна пам'ять Kingston Fury Renegade 32GB DDR5-6000",
    price: 5499,
    specifications: { ram_type: 'DDR5' }
  }
]
techstore>
```

6. Реалізував другу стратегію моделювання: Посилання через первинний ключ (References через ObjectId). У колекції 'orders' поле 'customer_id' містить 12-байтний ObjectId покупця, що усуває дублювання анкетних даних. Виконав два практичні запити: пошук усіх замовлень клієнта 'CUST-001' за його ObjectId, а також вибірку замовлень у черзі обробки ('status': 'processing'):

```
// замовлення з посиланням на покупця (customer_id: ObjectId) та масивом items
var custId = db.customers.findOne({ customer_code: "CUST-001" })._id;

db.orders.insertOne({
  order_number: "ORD-2026-101",
  customer_id: custId, // 12-байтний ObjectId покупця
  order_date: new Date("2026-03-10T10:30:00Z"),
  status: "completed",
  payment_method: "card",
  total_amount: 27498.0,
  items: [
    { product_id: prodRyzen, name: "Процесор AMD Ryzen 7 7800X3D", quantity: 1, unit_price: 18499.0 },
    { product_id: prodB650, name: "Материнська плата MSI MAG B650", quantity: 1, unit_price: 8999.0 }
  ]
});
```

```
techstore> var custId = db.customers.findOne({ customer_code: "CUST-001" })._id

techstore> db.orders.find({ customer_id: custId }, { order_number: 1, total_amount: 1, status: 1, cust
customer_id: 1, _id: 0 })
[
  {
    order_number: 'ORD-2026-101',
    customer_id: ObjectId('6ae39892841e7b6acfcb0ced'),
    status: 'completed',
    total_amount: 27498
  },
  {
    order_number: 'ORD-2026-107',
    customer_id: ObjectId('6ae39892841e7b6acfcb0ced'),
    status: 'processing',
    total_amount: 7899
  }
]
techstore> db.orders.find({ status: "processing" }, { order_number: 1, total_amount: 1, payment_method
payment_method: 1, customer_id: 1, _id: 0 })
[
  {
    order_number: 'ORD-2026-104',
    customer_id: ObjectId('6ae39892841e7b6acfcb0cf1'),
    payment_method: 'cash_on_delivery',
    total_amount: 13198
  },
  {
    order_number: 'ORD-2026-107',
    customer_id: ObjectId('6ae39892841e7b6acfcb0ced'),
    payment_method: 'card',
    total_amount: 7899
  }
]
techstore>
```

7. Реалізував третю стратегію: Серверне об'єднання зв'язаних колекцій за допомогою агрегаційного оператора '$lookup' та розгортання масиву '$unwind'. Відповідно до вимог методички виконав два окремі приклади агрегаційних конвеєрів:

Приклад 1: фільтрація завершених замовлень ('status': 'completed') з підтягуванням імені покупця та його міста проживання з колекції 'customers':

```
// агрегаційне з'єднання orders та customers за ключем customer_id -> _id
db.orders.aggregate([
  { $match: { status: "completed" } },
  {
    $lookup: {
      from: "customers",
      localField: "customer_id",
      foreignField: "_id",
      as: "customer_details"
    }
  },
  { $unwind: "$customer_details" },
  {
    $project: {
      _id: 0,
      order_number: 1,
      total_amount: 1,
      status: 1,
      customer_name: "$customer_details.name",
      customer_city: "$customer_details.address.city"
    }
  }
]);
```

```
techstore> db.orders.aggregate([
...   { $match: { status: "completed" } },
...   { $lookup: { from: "customers", localField: "customer_id", foreignField: "_id", as: "customer_de
tails" } },
...   { $unwind: "$customer_details" },
...   { $project: { _id: 0, order_number: 1, total_amount: 1, status: 1, customer_name: "$customer_det
ails.name", customer_city: "$customer_details.address.city" } }
... ])
[
  {
    order_number: 'ORD-2026-101',
    status: 'completed',
    total_amount: 27498,
    customer_name: 'Олександр Мельник',
    customer_city: 'Львів'
  },
  {
    order_number: 'ORD-2026-102',
    status: 'completed',
    total_amount: 49999,
    customer_name: 'Максим Шевченко',
    customer_city: 'Львів'
  },
  {
    order_number: 'ORD-2026-103',
    status: 'completed',
    total_amount: 13398,
    customer_name: 'Ірина Ковальчук',
    customer_city: 'Київ'
  },
  {
    order_number: 'ORD-2026-106',
    status: 'completed',
    total_amount: 31298,
    customer_name: 'Вікторія Мороз',
    customer_city: 'Київ'
  }
]
```

Приклад 2: вибірка великих замовлень на суму від 30 000 грн (total_amount >= 30000) із приєднанням контактного номера телефону покупця із колекції 'customers':

```
techstore> db.orders.aggregate([
...   { $match: { total_amount: { $gte: 30000 } } },
...   { $lookup: { from: "customers", localField: "customer_id", foreignField: "_id", as: "cust" } },
...   { $unwind: "$cust" },
...   { $project: { _id: 0, order_number: 1, total_amount: 1, customer: "$cust.name", phone: "$cust.ph
one" } }
... ])
[
  {
    order_number: 'ORD-2026-102',
    total_amount: 49999,
    customer: 'Максим Шевченко',
    phone: '+380933334455'
  },
  {
    order_number: 'ORD-2026-106',
    total_amount: 31298,
    customer: 'Вікторія Мороз',
    phone: '+380991234567'
  }
]
techstore>
```

Висновок:

Під час виконання лабораторної роботи №3 я опанував практичну роботу з нереляційною СКБД MongoDB в інтерактивній консолі mongosh. Дослідив клієнт-серверну архітектуру mongod/mongosh, повний життєвий цикл баз даних, створення звичайних та обмежених (capped) колекцій, механізми перейменування та вилучення даних. Реалізував базові CRUD-операції та практично відпрацював три підходи до моделювання зв'язків: вкладені документи (Embedded Documents) для швидкого читання специфікацій та адрес, нормалізовані посилання через ObjectId для замовлень, а також серверне об'єднання колекцій за допомогою агрегаційного конвеєра $lookup з оператором $unwind.